Marketing Workflow and Content Management Portal

Project Architecture and Complete Specification

MERN stack | Vercel + Render | Role-based | Content-centric tracking

Version	1.0
Date	13 October 2026
Based on	Media Manager codebase (partially developed) plus Zoho Projects feature reference
Audience	Developers, marketing leads, management

Contents

1. Project Overview 4

1.1 Objectives . . . 4

1.2 Design principles . . . 4

1.3 Starting point: the existing Media Manager . . . 4

2. Teams, Departments, and Work Sources 4

2.1 Production teams . . . 4

2.2 Requesting departments (work sources) . . . 4

3. Roles, Access, and Onboarding 5

3.1 Roles . . . 5

3.2 Common login and Request Access . . . 5

4. Content Tracking Model 5

4.1 Content Item fields . . . 6

4.2 Pipeline templates . . . 6

4.3 Statuses . . . 6

5. Content Calendar Module 6

6. Team Management Module 7

7. Feature Specification by Role 7

7.1 Admin . . . 7

7.2 Marketing Manager / Lead . . . 7

7.3 Team Lead (optional) . . . 7

7.4 Workers (Content, Graphics, Cinematography, Editor) . . . 8

7.5 Requesters . . . 8

7.6 Management / Viewer (optional) . . . 8

8. Reports and Analytics 8

8.1 Department reports . . . 8

8.2 Team reports . . . 8

8.3 Individual reports . . . 9

8.4 Content reports . . . 9

8.5 Work and bottleneck reports . . . 9

8.6 Management scorecard and tools . . . 9

8.7 Dashboards by role . . . 9

9. System Architecture 10

9.1 Layers . . . 10

9.2 Example flow: a reel from calendar to publish . . . 10

9.3 Real-time and background processing . . . 11

10. Technology Stack 11

10.1 Frontend . . . 11

10.2 Backend, data, and tooling . . . 11

10.3 What to avoid . . . 12
11. Repository and Module Structure 12
12. Data Model 12
13. API Reference (Planned) 13
13.1 Auth and access . . . 13
13.2 Requests, calendar, content . . . 13
13.3 Tasks, review, shoots, time . . . 14
13.4 Reports, notifications, admin data . . . 14
14. Workflow Rules, Automation, and Notifications 14
14.1 Transition rules (examples) . . . 15
14.2 Automation rules (examples) . . . 15
14.3 Notifications . . . 15
15. Security and Permissions 15
15.1 Permission keys . . . 15
15.2 Security checklist . . . 15
16. Deployment: Vercel and Render 16
16.1 Topology . . . 16
16.2 Cookies, CORS, and domains . . . 16
16.3 Vercel settings . . . 16
16.4 Render settings . . . 16
16.5 Email: Nodemailer now, Resend later . . . 17
16.6 Release checklist . . . 17
17. Fixes Required in the Current Codebase 17
18. Build Roadmap 18
19. Assumptions and Open Decisions 18

1. Project Overview

The portal is a central workspace for the marketing function. Other departments raise work requests, the social media team plans a monthly content calendar, and four production teams (Content, Graphics, Cinematography, Editor) deliver the work. Every piece of content is tracked from idea to publication, workers update status from their own portal, and management gets dashboards and reports on content, teams, departments, and individuals.

1.1 Objectives

- Give every department one place to request marketing work, and see its status without chasing people.
- Plan content from the social media calendar and turn it automatically into production work with deadlines.
- Assign work by team, role, skill, and current workload, and hand work between teams automatically.
- Let workers update status, upload files, and log time from a simple portal.
- Give the Marketing Lead and management live visibility of progress, risks, and bottlenecks.
- Produce content-based, work-based, department, team, and individual reports for decisions and reviews.

1.2 Design principles

- **Content-centric:** the Content Item (one post, reel, ad creative, brochure) is the main tracked unit; tasks hang under it.
- **Pipeline-driven:** each format has a template of stages; finishing a stage creates the next team's task.
- **Permission-based:** access is controlled by permissions, so roles can be adjusted without code changes.
- **History everywhere:** every status change is stored, which powers bottleneck and efficiency analytics.
- **Modular monolith:** one deployable API with clean modules; simple to host, easy to split later if needed.

1.3 Starting point: the existing Media Manager

The portal is built on the current Media Manager codebase (React 18 + Vite + TypeScript frontend, Express 5 + MongoDB + Socket.io backend, Cloudinary uploads). The table shows what is kept and what is new.

Area	Already in Media Manager (keep)	New or changed in the portal
Tasks	Kanban with dnd-kit, list view, subtasks, platform tags	Content Items, pipelines, handoffs, dependencies, time tracking
Review	Video review, approve / revision / reject, feedback comments	Versioned deliverables, timestamped comments, Lead approval, requester confirmation
Collaboration	Real-time task chat (Socket.io), typing indicators	Threaded comments with @mentions, activity feed
Access	JWT cookie auth, Google OAuth, admin / editor roles	Request Access flow, Admin approval, 6+ roles, permission middleware
Planning	Calendar page, attendance, team spaces	SM content calendar with import, capacity check, shoot planner
Reports	Basic Recharts charts	Department, team, individual, content, and bottleneck reports with export
Files	Multer to Cloudinary (500 MB, in-memory)	Direct signed uploads, file versions, file library

2. Teams, Departments, and Work Sources

2.1 Production teams

Team	Produces	Hands off to
Content	Scripts, captions, ad copy, blog and academic copy, brochure text	Graphics, Cinematography
Graphics	Posters, carousels, ad creatives, thumbnails, brochures, banners	Review
Cinematography	Shoots for reels, product, sales, and academic videos; footage	Editor
Editor	Video edits, reels, motion graphics, ad videos, final exports	Review

Each team has members with skills and weekly capacity, a leave calendar, a backlog queue, and a team workload view.

2.2 Requesting departments (work sources)

Every Content Item and request is tagged with its source department so reports can be cut by department.

Source	Typical work	Special fields captured
Social Media (SM Managers)	Monthly or weekly content calendar: posts, reels, stories, carousels	Publish date, platform, format, theme or campaign, caption brief, content pillar
Performance (Ads)	Ad creatives, A/B variants, creative refreshes	Ad platform, objective, formats and sizes, number of variants, audience, offer, launch date
Sales (Sales Lead)	Offer posters, brochures, product and pitch videos	Product or offer, target audience, urgency, delivery format
Academics (Academics Lead or Performance Lead)	Course creatives, webinar posters, academic videos	Program or course, event date, audience
Others	Any other marketing requirement	Generic form with type, description, deadline, references

Visibility rule: a department lead sees all requests of their department; other staff in the department see only their own requests.

3. Roles, Access, and Onboarding

3.1 Roles

Role	Purpose and scope
Admin	Overall control: approves access requests, manages users, roles, teams, master data, settings, audit log, and sees all reports. Can override in emergencies (always logged); not part of the normal approval flow.
Marketing Manager / Lead	Triages requests, plans and assigns work across all four teams, reviews and approves deliverables (replaces a separate Reviewer role), monitors workload and reports.
Team Lead (optional)	Manages one team's queue, assignment, and workload. Enabled only if each team has its own lead; otherwise the Marketing Lead manages all four teams.
Worker	Content writer, Graphic designer, Cinematographer, or Editor. Receives tasks, updates status, uploads work, logs time.
SM Manager	Requester who owns the content calendar, imports and edits calendar entries, and marks content as Published.
Department Lead	Performance, Sales, Academics, or other department head. Raises requests, tracks all requests of the department, confirms final output.
Department Staff	Requesters who see and manage only their own requests.
Management / Viewer (optional)	Read-only dashboards and reports for higher authorities.

Migration from the current code: **editor → worker**, **member → department staff**, new roles added.

3.2 Common login and Request Access

All users sign in through one common login page and are redirected to their role's home after login. New users click **Request Access** on the same page.

- **1.** User submits: name, work email, phone, department, requested role, skill type (for workers), reason. No password at this stage.
- **2.** The request is saved as Pending and the Admin is notified.
- **3.** Admin approves (can change role, team, skills, capacity), rejects with a reason, or asks for more information.
- **4.** On approval the account is created and an invite email with a set-password link (expires in 48 hours) is sent.
- **5.** The user sets a password and logs in. Pending or rejected users see a clear status message on login.

Access rules

- Only the Admin approves users; only the Admin can grant Lead or Admin roles.
- Block duplicate emails and repeat requests while one is pending; rate-limit and add captcha on the public request endpoint.
- Optional company email domain allowlist.
- Google OAuth logs in already-approved users only and never auto-creates accounts.
- Admin can suspend, reactivate, change role, reset password, and force logout.
- The first Admin is created by a one-time seed script using a password from the environment, never a default password.

4. Content Tracking Model

A **Content Item** is one deliverable: for example an Instagram reel due on 20 October, an ad creative set for a Meta campaign, or a sales brochure. Each Content Item owns a pipeline of tasks that move across the teams.

4.1 Content Item fields

Group	Fields
Identity	Content ID (e.g. CNT-2026-0142), title, format, campaign, content pillar
Source	Source department, requester, request ID (if raised as a request), calendar entry ID (if from the SM calendar)
Schedule	Publish date, production deadline (auto-calculated), priority, platform(s)
Production	Pipeline template, owner per stage, task links, estimates, status, hold reason
Assets	Script and brief, reference files, deliverables with versions, thumbnail, caption, hashtags
Publishing	Published link, published date, published by, optional post metrics
Audit	Full status history, comments, review cycles, revision count

4.2 Pipeline templates

When one stage is completed, the next stage's task is created and assigned automatically. Lead times are defaults and can be changed in master data.

Format	Pipeline	Default lead time
Static post or poster	Content (copy) → Graphics → Review → Approved → Scheduled → Published	3 days
Carousel	Content → Graphics → Review → Approved → Scheduled → Published	4 days
Story	Content → Graphics (or Editor) → Review → Approved → Scheduled → Published	2 days
Reel or video	Content (script) → Cinematography (shoot) → Editor → Review → Approved → Scheduled → Published	7 days
Ad creative	Content (ad copy) → Graphics or Editor → Review → Handed to Performance → Live	4 days
Brochure or sales material	Content → Graphics → Review → Delivered to Sales	5 days
Academic or webinar creative	Content → Graphics or Editor → Review → Delivered to Academics	4 days

4.3 Statuses

Level	Statuses
Request	Submitted → Under Triage → Accepted / Rejected / Needs Info → Converted to Content Item(s)
Content Item	Planned → Brief Ready → In Production → In Review → Changes Requested → Approved → Scheduled → Published (ads: Live / Paused / Retired)
Task	Assigned → In Progress → Submitted → Approved / Changes Requested → Done
Exceptions	**On Hold** and **Blocked** on any level, each with a required reason code (waiting for content, approval delay, resource shortage, client feedback, equipment, weather, other)

Statuses and transition rules are configurable in master data. Every transition is stored in StatusHistory with time, user, and reason.

5. Content Calendar Module

- Master calendar in month, week, and list views, filtered by platform, department, campaign, format, and team.
- SM managers add entries in the app or **bulk import from Excel or CSV**. The approved calendar automatically creates Content Items and their pipelines.
- **Production deadline is auto-calculated** as publish date minus the format's lead time; the calendar shows both dates.
- **Capacity check:** the system compares demand with team capacity, for example 12 reels due next week against capacity for 8, and flags the gap before the week starts.
- Clash detection for too many posts on one day or platform; at-risk items highlighted in amber or red.
- SM manager marks an item Published with link and date; post metrics (reach, views, likes) can be entered, and later pulled from platform APIs.
- Calendar overlays: shoots, milestones, leave, and holidays.

6. Team Management Module

Capability	Details
Team pages	Members, skills, weekly capacity, today's load, leave, current tasks, queue, and team metrics for each of the four teams
Assignment	Suggested assignee by team, skill, and current load; leave-aware; overload warnings; reassign by drag-and-drop; optional auto-assignment (type → team → least loaded)
Handoff queues	Editor sees Footage Ready; Graphics sees Copy Ready; Cinematography sees Script Ready; the Lead sees Waiting for Review
Shoot Planner	Group several video Content Items into one shoot day with location, crew, equipment, talent, and a script-ready check. Statuses: Scheduled → Done → Reshoot Needed. Completing a shoot releases footage to the Editor queue
Workload view	Per person: open tasks, planned versus capacity hours, leave; weekly capacity planner
Skills matrix	Skill tags per person (e.g. reels, motion graphics, carousel design, Malayalam copy) used for suggestions
Attendance and leave	Present or leave status feeds assignment and capacity; leave calendar visible to Leads
Timesheets	Timer or manual hours per task, weekly timesheet, Lead approval

7. Feature Specification by Role

7.1 Admin

Menu: Dashboard, Access Requests, Users, Teams and Departments, Master Data, Reports, Audit Log, Settings

- **Dashboard:** pending access requests (badge), users by role, active versus suspended, system-wide requests, content and tasks, company-wide on-time rate, recent activity.
- **Access requests:** review, approve (edit role, team, skills, capacity), reject with reason, ask for info, bulk approve.
- **Users:** view, edit, change role, suspend or reactivate, reset password, force logout, deactivate.
- **Teams and departments:** create teams, assign Leads and members, map requester departments.
- **Master data:** request types and dynamic fields, formats and pipeline templates, lead times, statuses, priorities, hold reasons, platforms, SLAs.
- **Settings:** permission sets per role, email templates, digest timing, efficiency score weights.
- **Audit log:** role changes, deletions, approvals, status overrides; filter and export.
- **Override:** reassign, change status, or approve in emergencies (logged). **Reports:** all reports, exports, scheduling.

7.2 Marketing Manager / Lead

Menu: Dashboard, Request Inbox, Content Calendar, Content Items, Tasks, Workload, Shoot Planner, Review Queue, Reports, Team

- **Dashboard:** requests awaiting triage, content due today or this week, overdue, blocked or on hold, awaiting review, team workload, pipeline by stage, at-risk publishes.
- **Request inbox:** accept, reject with reason, ask for information, set priority and deadline, merge duplicates, convert to Content Item(s) from a pipeline template, bulk triage.
- **Planning:** approve the SM calendar, review the capacity check, adjust lead times and deadlines, plan shoots.
- **Assignment:** assign or reassign with skill and load shown; collaborators; estimates; dependencies; re-prioritize (requester notified).
- **Review and approval:** review queue by deadline; video player with timestamped comments; version comparison; approve, request changes, or reject; send to requester for final confirmation; revision count.
- **Team:** worker list with skills, workload, attendance; timesheet approval; individual performance summaries.
- **Reports:** department, team, individual, content, and bottleneck reports.
- **Cannot:** manage users or roles, approve access requests, change system settings.

7.3 Team Lead (optional)

- Same as Lead but scoped to one team: team queue, assignment inside the team, team workload, team reports, timesheet approval for the team.
- May review work if the Lead delegates review to them (permission flag).

7.4 Workers (Content, Graphics, Cinematography, Editor)

Menu: My Dashboard, My Tasks, Calendar, Timesheet, Files, Profile

- **Dashboard:** today's tasks, due this week, overdue, blocked, hours logged, pending revisions, personal stats (completed, on-time %, first-pass approval).
- **My tasks:** board, list, and calendar views; filters by priority, platform, due date; full brief, references, subtasks, comments.
- **Status:** Assigned → In Progress → Submitted → Changes Requested → Done; mark Blocked or On Hold with a required reason (alerts the Lead); request a deadline extension.
- **Delivery:** upload versioned files (video up to 500 MB, via direct upload), attach external links, submit for review with notes, read feedback and fix.
- **Time and attendance:** start/stop timer or manual entry, weekly timesheet, present or on leave.

Team	Team-specific features
Content	Script and caption editor (rich text), brief view, caption and hashtag fields, copy versions, language tag
Graphics	Size and format specs per platform, asset upload with preview, carousel slide order, brand asset links
Cinematography	Shoot list for the day, script and shot list view, equipment and location notes, mark shoot Done or Reshoot, upload raw footage links
Editor	Footage Ready queue, script and reference view, export specs, versioned exports, submit for review with timestamped notes

Cannot: assign tasks, create projects, approve work, see other workers' stats or other teams' tasks, open admin pages.

7.5 Requesters

Requester	Menu and features
SM Manager	Content Calendar (create, edit, bulk import), calendar status dashboard (planned, in production, ready, published), at-risk posts, mark Published with link, post metrics, approve final output
Performance (Ads)	New ad creative request (platform, objective, sizes, variants, audience, launch date), creative status board (requested, in production, ready, live), confirm or request changes, creative refresh requests
Sales Lead	New request (offer, product, audience, urgency), department request list, confirm deliverables, download approved files
Academics Lead / Performance Lead	New request (course, event date), department request list, confirm deliverables, download approved files
Others / Department Staff	New request, My Requests with status timeline, comments to Lead, cancel before work starts, edit and resubmit if rejected, confirm final output

Requesters never see internal tasks, worker workload, or other departments' requests.

7.6 Management / Viewer (optional)

Read-only dashboards and the complete report set with filters, drill-down, export, and scheduled emails.

8. Reports and Analytics

8.1 Department reports

For Social Media, Performance, Sales, Academics, and Others.

- Requests and Content Items by type, status, and month or week.
- Turnaround time (request → delivered) and on-time delivery rate.
- Revision rate: how often a department's requests need rework.
- Pending and overdue items, with aging.
- Effort consumed: hours spent per department, per team, and per format (supports allocation decisions).

8.2 Team reports

For Content, Graphics, Cinematography, and Editor.

- Throughput and backlog by week and month.
- Queue time versus work time; average turnaround by format.

- Capacity utilization (logged hours versus capacity).
- Rework rate and handoff delays, for example the Editor waiting on shoots.

8.3 Individual reports

- Tasks assigned versus completed; on-time delivery rate.
- Average time per task type; actual versus estimated hours.
- **First-pass approval rate** and revision count.
- Hours logged versus capacity; leave days; workload trend.
- **Efficiency score** (suggested default, weights configurable by Admin): on-time delivery 40%, first-pass approval 30%, speed versus estimate 20%, volume and utilization 10%.
- Requester-side view: requests raised, revisions requested, average response time on approvals.

8.4 Content reports

- Output by format, platform, week or month, campaign, content pillar, and source department.
- **Planned versus published** (calendar adherence) and on-time publish rate.
- Pipeline by stage: how many items are sitting in each stage.
- Late content list and aging; average review cycles per format.
- Ad creatives delivered per performance campaign; creative refresh frequency.
- Optional post performance (reach, views, engagement) by format and platform, once metrics are captured.

8.5 Work and bottleneck reports

- **Time in each status** for requests, Content Items, and tasks (built from StatusHistory).
- **Where work holds or crashes:** hold and block counts by reason code, stage, team, and department.
- Handoff delay between teams (for example Cinematography → Editor, Content → Graphics).
- Rework reasons and rejection reasons; SLA breaches; overdue aging.

8.6 Management scorecard and tools

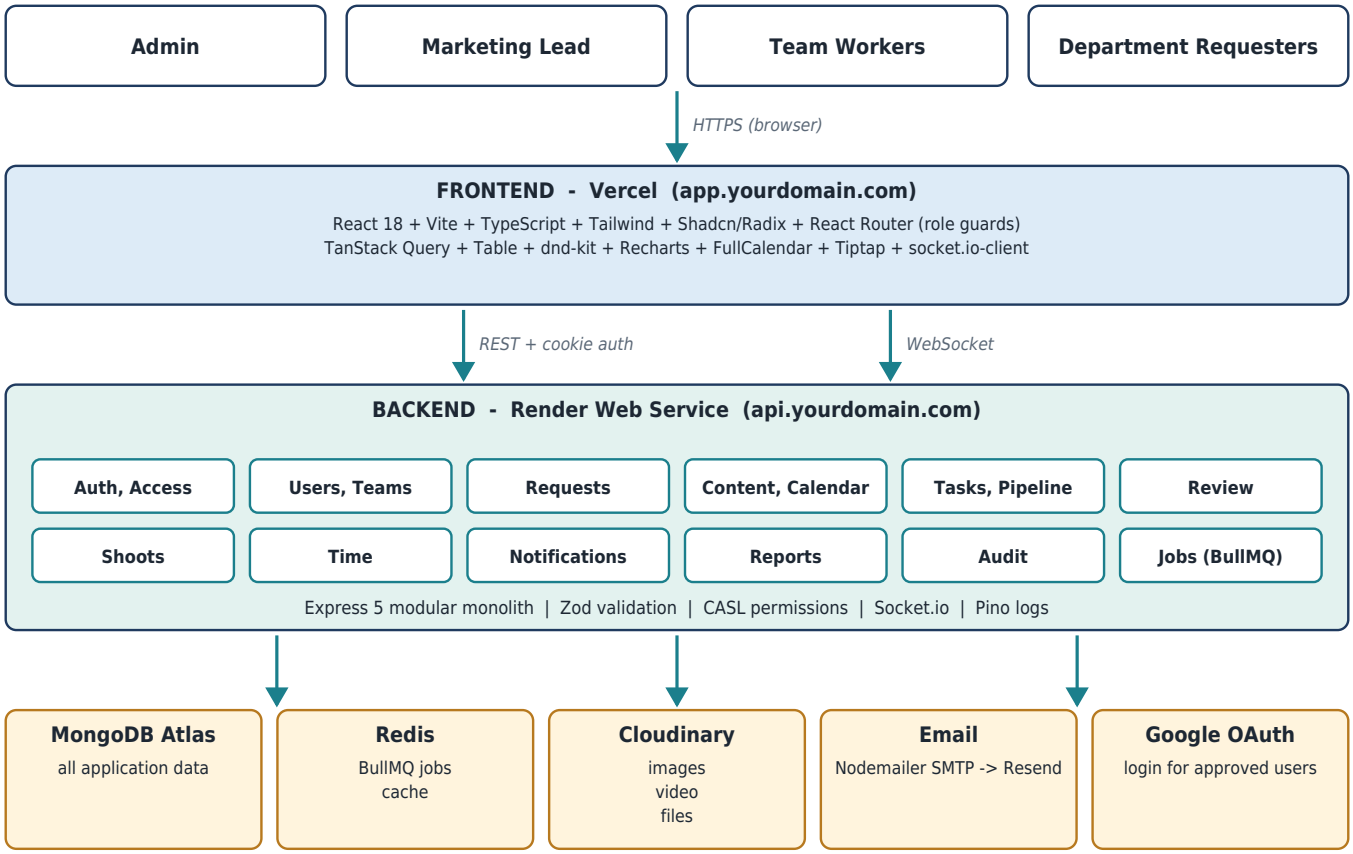
- Monthly scorecard: output, on-time rate, first-pass approval, utilization, backlog; month-over-month trend.
- Demand versus capacity forecast for the coming weeks; top performers and problem areas.
- All reports: date range, department, team, person, format, platform filters; drill-down; saved reports; export to PDF, Excel, CSV; scheduled email delivery.

8.7 Dashboards by role

Role	Key widgets
Management	Scorecard, on-time %, project and content health, department demand, bottlenecks, utilization
Lead	Pipelines by stage, workload per team, awaiting triage, awaiting review, blocked and at-risk items, shoots this week
SM Manager	Calendar status, at-risk posts this week, publishing queue, capacity warnings
Performance	Ad creatives requested, in production, ready, live
Sales and Academics leads	Department requests by status, upcoming deliveries
Workers	My queue, today's tasks, handoffs waiting, hours this week, personal stats
Admin	Pending access requests, user counts, system-wide activity, SLA summary

9. System Architecture

A decoupled web application: a React single-page app on Vercel talks to an Express API on Render over REST and WebSocket. Data lives in MongoDB Atlas; files in Cloudinary; background jobs in Redis queues.



9.1 Layers

Layer	Responsibility	Technology
Client	Role-based UI, forms, boards, calendar, charts, review player, real-time updates	React 18, Vite, TypeScript, Tailwind, Shadcn/Radix
API	Authentication, permissions, business rules, pipelines, reports, file signing, email	Node.js LTS, Express 5, Zod, CASL
Real-time	Task chat, typing, live status and notification pushes, per-room channels	Socket.io (Redis adapter if scaled)
Jobs	Reminders, daily digests, scheduled reports, recurring requests, email sending	BullMQ + Redis
Data	Application data, status history, time entries, audit log	MongoDB Atlas, Mongoose
Files	Images, video, documents, versions	Cloudinary with signed direct upload
Email	Invites, notifications, digests	Nodemailer now; Resend later (adapter)

9.2 Example flow: a reel from calendar to publish

- SM manager imports the October calendar. The entry for a reel on 20 October creates a Content Item with a Reel pipeline; the production deadline is set to 13 October.
- Content task (script) is assigned to a writer. On submit and approval, a Cinematography task appears in the Shoot Planner queue.
- The Lead groups it into a shoot day. When the shoot is marked Done, footage is released to the Editor queue and a task is created.

- Editor uploads version 1 and submits. The Lead reviews with timestamped comments, requests changes; the Editor uploads version 2; the Lead approves.
- The requester confirms; the item becomes Scheduled; the SM manager publishes and enters the link. Every step is stored in StatusHistory and appears in reports.

9.3 Real-time and background processing

- Socket.io rooms per task and per user: chat, typing, status changes, and notification pushes. JWT is verified in the socket handshake.
- BullMQ jobs: deadline reminders, overdue alerts, daily digest, scheduled report emails, recurring request generation, invite and notification emails.
- Email sending is queued so slow SMTP calls never block API responses.

10. Technology Stack

10.1 Frontend

Need	Choice	Status
Framework	React 18, Vite, TypeScript	Keep
Styling and UI	Tailwind CSS, Shadcn/Radix	Keep
Server state	TanStack Query	Keep
Forms and validation	React Hook Form, Zod (shared schemas)	Keep
Routing	React Router with role-based guards	Keep
Kanban	dnd-kit	Keep
Charts	Recharts	Keep
Real-time	socket.io-client	Keep
Data tables	TanStack Table	Add
Calendar	FullCalendar	Add
Gantt (later)	gantt-task-react or DHTMLX	Add (Phase 5)
Video review	Video.js or Plyr with custom timestamp comments	Add
Rich text	Tiptap (briefs, scripts, mentions)	Add
Uploads	react-dropzone with progress	Add
Light client state	Zustand (only if Context becomes messy)	Optional

10.2 Backend, data, and tooling

Need	Choice	Status
Runtime and framework	Node.js LTS (22), Express 5	Keep (move off Node 25)
Database	MongoDB Atlas, Mongoose	Keep
Validation	Zod, shared package with the frontend	Add
Permissions	CASL (permission-based access)	Add
Auth	JWT in httpOnly cookies, short-lived access plus refresh token, Google OAuth for approved users	Keep and harden
Jobs and cache	BullMQ and Redis	Add
Email	Nodemailer (SMTP) now, Resend later through an adapter	Add
Security	helmet, express-rate-limit, express-mongo-sanitize, hpp	Add
Logging and errors	pino, Sentry	Add
API docs	Swagger / OpenAPI	Add
Exports	ExcelJS (Excel, CSV), pdfmake or Puppeteer (PDF)	Add
Files	Cloudinary with signed direct upload	Keep and change flow
Testing	Vitest or Jest, Supertest, React Testing Library, Playwright	Add

Need	Choice	Status
Quality and CI	ESLint, Prettier, Husky, GitHub Actions	Add
Containers	Docker and docker-compose for local development	Add

10.3 What to avoid

- Microservices (a modular monolith is enough for one marketing function).
- GraphQL (REST plus TanStack Query is simpler here).
- Next.js (an internal portal has no SEO need).
- Redux (TanStack Query already covers server state).
- Building a Gantt chart or calendar from scratch.

11. Repository and Module Structure

```
marketing-portal/
├── apps/
│   ├── api/                        # Express 5 API (deploys to Render)
│   │   └── src/
│   │       ├── server.js           # http server, Socket.io, middleware
│   │       ├── config/             # env, db, redis, cloudinary
│   │       ├── lib/                # mailer, permissions (CASL), logger, queue
│   │       ├── middleware/         # auth, permission, validate, error
│   │       ├── jobs/               # BullMQ workers: reminders, digests, reports
│   │       └── modules/
│   │           ├── auth/           # login, set-password, refresh, google
│   │           ├── access/         # request-access, admin approval
│   │           ├── users/          teams/   departments/
│   │           ├── requests/       # intake per source department
│   │           ├── content/        # content items, pipelines, publishing
│   │           ├── calendar/       # entries, import, capacity check
│   │           ├── tasks/          # tasks, subtasks, dependencies, handoffs
│   │           ├── review/         # deliverables, versions, comments, approval
│   │           ├── shoots/         # shoot planner
│   │           ├── time/           # timesheets
│   │           ├── notifications/  # in-app, email, digest
│   │           ├── reports/        # aggregations, exports, scheduled
│   │           └── audit/
│   └── web/                        # React + Vite (deploys to Vercel)
│       └── src/
│           ├── pages/{admin,lead,worker,requester,shared}/
│           ├── components/  hooks/  context/  lib/
│           └── App.tsx             # role-based routes
├── packages/shared/                # Zod schemas, roles, permission keys, types
├── docker-compose.yml              # api, mongo, redis for local dev
└── .github/workflows/              # CI
```

Each API module contains its own **routes, controller, service, model, and schema** files. Controllers stay thin; business rules live in services, which makes pipelines and reports testable.

12. Data Model

Collections in MongoDB. References use ObjectId (the current code has some string and email references to clean up).

Collection	Key fields	Notes
User	name, email, passwordHash, role, status (active/suspended), department, teamId, skills[], capacity, attendance, mustSetPassword, lastLogin	Extends existing model
AccessRequest	name, email, phone, department, requestedRole, skills[], reason, status, reviewedBy, reviewedAt, assignedRole, rejectReason, inviteTokenHash, inviteExpiresAt	New
Team	name (Content, Graphics, Cinematography, Editor), lead, members[], capacity settings	Extends existing
Department	name, lead, type (SM, Performance, Sales, Academics, Other)	New
Request	requestId, source department, requester, type, dynamic fields, priority, deadline, attachments[], status, triage info, linked content items	New
CalendarEntry	date, platform, format, theme, campaign, caption brief, owner, import batch, linked content item	New
Campaign	name, department, dates, platforms, goals	New

Collection	Key fields	Notes
ContentItem	contentId, title, format, source department, requester, campaignId, pillar, platforms, publishDate, productionDeadline, status, pipeline stages, caption, publishedUrl, metrics	New, central
PipelineTemplate	format, stages[] (team, name, lead time, default role), approval rules	New, configurable
Task	contentItemId, stage, team, assignee, collaborators, status, estimate, actualHours, dependencies[], holdReason, revisionCount, due date	Extends existing
Deliverable	taskId, contentItemId, version, fileUrl, uploadedBy, status, reviewDecision	Merges Submission and Content
Comment	entityType, entityId, author, text, mentions[], timestamp (for video notes), attachments[]	New; chat stays in Message
Message	taskId, sender, text, timestamp	Existing, keep
Shoot	date, location, crew[], equipment, contentItems[], status, notes	New
TimeEntry	taskId, userId, start, end, minutes, note, approvedBy	New
StatusHistory	entityType, entityId, fromStatus, toStatus, changedBy, reason, timestamp	New, powers analytics
Notification	userId, type, message, entity link, read, createdAt	Existing, extended
AutomationRule	trigger, conditions, actions, active	New (later phase)
AuditLog	actor, action, entity, before, after, timestamp, ip	New

Recommended indexes

- StatusHistory: { entityType, entityId, timestamp } and { toStatus, timestamp }.
- Task: { assignee, due date }, { team, status }, { contentItemId, stage }.
- ContentItem: { status, publishDate }, { sourceDepartment, publishDate }, { campaignId }.
- TimeEntry: { userId, start }, { taskId }. Request: { sourceDepartment, status, createdAt }.

13. API Reference (Planned)

All routes sit under /api and are protected by authentication plus a permission check. Access column shows who may call the route in the normal flow.

13.1 Auth and access

Method	Endpoint	Access	Description
POST	/auth/login	Public	Common login for all roles; sets httpOnly cookie
POST	/auth/logout, /auth/refresh	Auth	Clear session, refresh access token
GET	/auth/me	Auth	Current user, role, permissions
POST	/auth/request-access	Public (rate-limited)	Submit access request
POST	/auth/set-password	Invite token	Set password from invite link
GET	/auth/google, /auth/google/callback	Public	Google sign-in for approved users
GET	/admin/access-requests?status=	Admin	List access requests
PATCH	/admin/access-requests/:id/approve	Admin	Approve with role, team, skills; sends invite
PATCH	/admin/access-requests/:id/reject	Admin	Reject with reason
GET / PATCH	/admin/users, /admin/users/:id/(role|suspend|reset-password)	Admin	User management

13.2 Requests, calendar, content

Method	Endpoint	Access	Description
POST / GET	/requests	Requester, Lead	Create and list requests (scoped by department)
PATCH	/requests/:id/triage	Lead	Accept, reject, ask info, convert to content items

Method	Endpoint	Access	Description
GET / POST	/calendar/entries	SM Manager, Lead	List and create calendar entries
POST	/calendar/import	SM Manager	Bulk import from Excel or CSV
GET	/calendar/capacity?from=&to;=	Lead, SM Manager	Demand versus capacity check
GET / POST	/content	Role-scoped	List and create content items
GET / PATCH	/content/:id	Role-scoped	Detail and update
PATCH	/content/:id/status	Role rules	Transition with reason where required
PATCH	/content/:id/publish	SM Manager	Mark published with link and date
GET / PUT	/pipeline-templates	Admin, Lead	View and edit pipeline templates

13.3 Tasks, review, shoots, time

Method	Endpoint	Access	Description
GET	/tasks, /tasks/mine, /tasks/queue/:team	Role-scoped	Lists, personal tasks, handoff queue
POST	/tasks/:id/assign	Lead, Team Lead	Assign or reassign
PATCH	/tasks/:id/status	Assignee, Lead	Status change; hold reason required for Blocked
POST	/tasks/:id/deliverables	Assignee	Register uploaded file version
POST	/uploads/sign	Auth	Signed parameters for direct Cloudinary upload
POST	/review/:deliverableId/decision	Lead	Approve, request changes, reject
POST	/requests/:id/confirm	Requester	Final confirmation or change request
GET / POST	/comments?entity=	Role-scoped	Threaded comments, video timestamps
GET / POST / PATCH	/shoots	Lead, Cinematography	Plan shoots, mark done or reshoot
POST / GET	/time-entries	Worker, Lead	Log and list time; approve timesheets
GET	/workload?team=&week;=	Lead	Per-person load versus capacity

13.4 Reports, notifications, admin data

Method	Endpoint	Access	Description
GET	/dashboard/:role	Role	Role-specific dashboard summary
GET	/reports/department	Admin, Lead, Dept Lead (own)	Department report
GET	/reports/team	Admin, Lead, Team Lead (own)	Team report
GET	/reports/individual/:userId	Admin, Lead, self	Individual performance
GET	/reports/content	Admin, Lead, SM Manager	Content output and adherence
GET	/reports/bottlenecks	Admin, Lead, Management	Time in status, holds, handoff delay
GET	/reports/export?format=xlsx|csv|pdf	As report	Export any report
POST / GET	/reports/schedules	Admin, Lead	Scheduled email reports
GET / PATCH	/notifications	Auth	List and mark read
GET / PUT	/admin/master-data/:type	Admin	Request types, statuses, reasons, platforms, SLAs
GET	/admin/audit-log	Admin	Audit trail with filters
GET	/status	Public	Health check for Render

14. Workflow Rules, Automation, and Notifications

14.1 Transition rules (examples)

- A task cannot move to Submitted without at least one uploaded deliverable (or link) and, for editors, export notes.
- Blocked or On Hold requires a reason code; the Lead is notified immediately.
- Only the Lead (or a delegated reviewer) can Approve or Request Changes; only the requester can confirm final output.
- A Content Item cannot be Scheduled before it is Approved and, if the source is a request, confirmed by the requester.
- Only the SM Manager can set Published; only the Performance team can set an ad Live.
- A dependent task cannot start until the task it depends on is done.

14.2 Automation rules (examples)

- Stage completed → create and assign the next stage's task from the pipeline template.
- Request accepted → create Content Items and tasks from the template.
- Deadline in 24 hours → notify assignee; overdue by 1 day → notify the Lead.
- Shoot marked Done → release footage to the Editor queue.
- Weekly and monthly recurring requests are generated by a scheduled job.
- Capacity gap detected for next week → alert the Lead and SM Manager.

14.3 Notifications

Audience	Notified when
Admin	New access request, SLA breach summary, user suspended, system alerts
Lead	New request, task blocked, deadline at risk, submission ready for review, capacity gap, requester feedback
Worker	New assignment, handoff ready, deadline tomorrow or overdue, review feedback, mention, re-prioritized
Requester	Request accepted or rejected, info needed, status change, ready for confirmation, delivered, published

Channels: in-app (live via Socket.io), email, daily digest for Leads and management; WhatsApp or Slack can be added later.

15. Security and Permissions

15.1 Permission keys

Routes and UI actions check permission keys through CASL. A role is a set of permissions, so adding or adjusting roles does not require code changes.

Permission key	Admin	Lead	Team Lead	Worker	SM Mgr	Dept Lead	Dept Staff
user.approve / user.manage	✓						
settings.manage / audit.view	✓						
request.create	✓	✓			✓	✓	✓
request.view.department	✓	✓			✓	✓	
request.triage	✓	✓					
calendar.manage / import	✓	✓			✓		
content.create / update	✓	✓	✓		✓		
content.publish	✓				✓		
task.assign	✓	✓	team				
task.update.own / upload	✓	✓	✓	✓			
review.approve	override	✓	if delegated				
request.confirm					✓	✓	✓
shoot.manage	✓	✓	✓	Cinema			
time.log				✓			
timesheet.approve	✓	✓	team				
report.view	all	all	team	own	calendar	department	own

15.2 Security checklist

- JWT in httpOnly, Secure cookies; short-lived access token with refresh token; CSRF protection or SameSite policy; rotate secrets.
- Remove public registration and the task seed endpoint in production; no default passwords; strong secrets in environment variables only.
- Permission middleware on every route (many current routes are only 'Authenticated', including delete team and delete task).
- Input validation with Zod on every body, query, and param; mongo sanitization; helmet; rate limits on login and request-access.
- Department scoping enforced in queries, not only in the UI.
- Signed direct uploads with type and size limits; no large buffers in server memory.
- Audit log for role changes, approvals, deletions, and overrides; invite and reset tokens stored hashed and expiring.
- Backups (Atlas), error monitoring (Sentry), and structured logs without secrets.

16. Deployment: Vercel and Render

16.1 Topology

Component	Host	Notes
React app (apps/web)	Vercel	app.yourdomain.com; SPA rewrite so deep links work
Express + Socket.io (apps/api)	Render Web Service	api.yourdomain.com; health check /api/status; PORT from Render
Database	MongoDB Atlas	Allow Render outbound IPs; strong DB user
Redis (jobs, cache)	Render Key Value or Upstash	Needed from the reports and reminders phase
Worker process (later)	Render Background Worker	Runs BullMQ jobs separately from the API
Files	Cloudinary	Signed direct browser upload

16.2 Cookies, CORS, and domains

Auth uses httpOnly cookies, and the frontend and API are on different hosts, so use **one custom domain with subdomains** (app. and api.). Third-party cookie blocking in browsers such as Safari breaks cross-site cookies between vercel.app and onrender.com. Do not proxy the API through Vercel rewrites, because Socket.io WebSockets will not pass; the frontend connects to Render directly.

```
app.set('trust proxy', 1);                    // secure cookies behind Render's proxy
app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));
const io = new Server(server, { cors: { origin: process.env.FRONTEND_URL, credentials: true } });
res.cookie('token', jwt, { httpOnly: true, secure: true, sameSite: 'lax', domain: '.yourdomain.com' });
const PORT = process.env.PORT || 8080;        // Render injects PORT
```

16.3 Vercel settings

- Root directory apps/web, framework Vite, build command npm run build, output dist.
- Environment: VITE_API_BASE_URL=https://api.yourdomain.com/api and VITE_SOCKET_URL=https://api.yourdomain.com.
- vercel.json rewrite: { "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }.

16.4 Render settings

- Web Service, root directory apps/api, build npm install, start node src/server.js, health check /api/status, NODE_VERSION=22.
- Free instances sleep when idle: first request is slow, Socket.io connections drop, and scheduled jobs do not run reliably. Use a paid instance for real team use.

Environment variable	Purpose
NODE_ENV, PORT	production mode; port injected by Render
MONGODB_URI	MongoDB Atlas connection string
JWT_SECRET, JWT_REFRESH_SECRET	Long random secrets (rotate)
FRONTEND_URL, COOKIE_DOMAIN	CORS origin and cookie domain
CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET	Signed uploads

Environment variable	Purpose
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI	Google sign-in
REDIS_URL	BullMQ and cache (when enabled)
EMAIL_PROVIDER (smtp or resend), MAIL_FROM	Mail adapter switch and sender address
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS	Nodemailer SMTP settings
RESEND_API_KEY	Used after switching to Resend
ADMIN_SEED_EMAIL, ADMIN_SEED_PASSWORD	One-time first Admin creation

16.5 Email: Nodemailer now, Resend later

Important: Render blocks outbound SMTP ports 25, 465, and 587 on free web services (since 26 September 2025), and port 25 stays blocked on every plan. Nodemailer over SMTP therefore needs a **paid Render instance** with port 587 or 465. On a free instance, use an HTTPS-based sender. To make the later switch painless, all mail goes through one adapter that is selected by EMAIL_PROVIDER.

```
// lib/mailer.js
import nodemailer from 'nodemailer';
const provider = process.env.EMAIL_PROVIDER || 'smtp';
const transporter = provider === 'smtp' ? nodemailer.createTransport({
  host: process.env.SMTP_HOST, port: Number(process.env.SMTP_PORT || 587),
  secure: Number(process.env.SMTP_PORT) === 465,
  auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS } }) : null;

export async function sendMail({ to, subject, html, text }) {
  const from = process.env.MAIL_FROM;
  if (provider === 'smtp') return transporter.sendMail({ from, to, subject, html, text });
  if (provider === 'resend') {
    const { Resend } = await import('resend');
    return new Resend(process.env.RESEND_API_KEY).emails.send({ from, to, subject, html, text });
  }
  throw new Error('Unknown EMAIL_PROVIDER: ' + provider);
}
```

Switching to Resend later: install the resend package, verify the sending domain, set EMAIL_PROVIDER=resend and RESEND_API_KEY. No other code changes.

16.6 Release checklist

- Create Atlas cluster and user; deploy the API; confirm /api/status.
- Deploy the web app with API and socket URLs; attach custom domains; set FRONTEND_URL and cookie domain.
- Run the one-time Admin seed; test login, Request Access, invite email, set-password, chat, and upload.
- Enable Redis and the worker when reminders, digests, and scheduled reports go live.
- Turn on Sentry, uptime checks, and Atlas backups before the team starts using it.

17. Fixes Required in the Current Codebase

Issue found in Media Manager	Risk	Fix
Many routes only require 'Authenticated' (delete team, create team, delete task, seed)	Any logged-in user can destroy data	Permission middleware per route; remove seed route in production
Public /api/auth/register	Uncontrolled signups	Replace with Request Access flow
No Content Item, Project, or Request entities	Cannot track content or intake	Add ContentItem, Request, Campaign, pipelines
No status history	Cannot find bottlenecks	Add StatusHistory for every transition
Comments embedded in Task and separate chat Message	Two comment systems	Keep chat for live talk; separate Comment collection
Submission and Content overlap	Duplicate review data	Merge into Deliverable with versions
Feedback.userId string, Submission.user_email	Weak references	Use ObjectId references
Multer buffers up to 500 MB in memory	Crashes under parallel uploads, esp. on small instances	Signed direct browser-to-Cloudinary upload

Issue found in Media Manager	Risk	Fix
Default seeded credentials and sample JWT secret in docs	Credential exposure	Strong secrets, no default Admin password, rotate
Node 25 in use	Not an LTS release	Use Node 22 LTS
Fixed 6 task statuses, single assignee	Limits real workflows	Configurable statuses, collaborators, pipeline stages

18. Build Roadmap

Phase	Scope	Outcome
1. Foundation	Permission middleware and CASL, new roles, Request Access and Admin approval, Departments and Teams, security fixes, Node LTS, mail adapter	Secure base with controlled onboarding
2. Intake and content core	Request module per source department, Lead inbox, ContentItem, pipeline templates and handoffs, StatusHistory, Deliverable and versions	Requests become tracked content moving across teams
3. Calendar and teams	SM content calendar with import and auto deadlines, capacity check, team pages, workload view, handoff queues, Shoot Planner, signed uploads	Monthly planning and production coordination
4. Review and time	Timestamped video review, Lead approval, requester confirmation, publishing queue, time tracking and timesheets, notifications and digests, Redis and BullMQ	Full delivery loop with accountability
5. Reports	Department, team, individual, content, and bottleneck reports, dashboards per role, efficiency score, exports, scheduled emails	Management visibility and performance tracking
6. Advanced	Gantt, automation rule builder, custom fields, post metrics from platform APIs, Slack or WhatsApp alerts, Resend migration	Polish and scale

19. Assumptions and Open Decisions

Assumptions made in this document

- The Marketing Manager / Lead manages all four teams. The Team Lead role is optional and can be enabled later.
- Reviewer and Approver duties belong to the Lead; the Admin is a separate overall role.
- SM managers own the content calendar and are the only ones who mark content as Published.
- Lead times per format, status lists, and the efficiency score weights are defaults that Admin can change.
- Deployment uses Vercel for the frontend and Render for the API, with Nodemailer first and Resend later.

Decisions to confirm

- Does each team (Content, Graphics, Cinematography, Editor) have its own team lead?
- Can requesters see the assigned worker's name, or only the status and the Lead?
- Should the Performance team's ad results (which creative worked) be recorded back against each creative?
- Is post performance (reach, views) in scope now, or only publication tracking?
- Final efficiency score formula and report access for higher authorities.
- Which Render plan will be used (paid is required for SMTP email and reliable background jobs)?